

LAB1 2024-2

1.- (4 puntos) Escriba un *shell script* (**script1.sh**) que reciba dos números, n_1 y n_2 . El *script* debe imprimir todos los números entre n_1 y n_2 . Solo en caso que el número sea primo deberá imprimir el número y a continuación el mensaje: (primo).

Version NICK:

```
#!/bin/bash
## Pregunta01
# Lectura de datos
while true # Comprueba que no esten vacios
do
    echo "Ingrese 2 valores numericos"
    read n1 n2
    if [[ -z $n1 || -z $n2 ]]; then #compara que no sea 0
        echo "Debe ingresar los 2 valores"
    else
        #compara si es un numero
        if [[ $n1 =~ ^[0-9]+$ && $n2 =~ ^[0-9]+$ ]]; then
            break
        else
            echo "Deben ser numericos"
        fi
    fi
done
# Comprobar el numero menor (n1 debe ser el menor)
if [ $n1 -ge $n2 ]; then
    temp=$n1
    n1=$n2
    n2=$temp
fi
# Funcion para comprobar si es divisible
esDivisible(){
    modulo=$(( $1 % $2 ))
    if [ $modulo -eq 0 ]; then
        return 0 #true
    else
        return 1
    fi
}
# Funcion para comprobar si un numero es primo
comprobarPrimo(){
    start=2
    while [ $start -lt $1 ]
    do
        if esDivisible $1 $start; then
            return 1
        fi
        ((start++))
    done
    return 0
}
# Funcion de impresion programa

while [[ $n1 -le $n2 ]]
do
    echo -n "$n1 "
    if comprobarPrimo $n1; then
        echo -n "primo"
    fi
    echo ""
    ((n1++))
done
```

Version GH:

```
esPrimo() {
    local num=$1
    if [[ $num -lt 2 ]]; then
```

```

        return 1
    fi
    for ((i=2; i*i<=num; i++)); do
        if ((num % i == 0)); then
            return 1
        fi
    done
    return 0
}

# Verificar si la cantidad de mi INPUT es 2:
if [ $# -ne 2 ]; then
    echo "Uso: $0 n1 n2"
    exit 1
fi

#Ingreso estos valores:
n1=$1
n2=$2

for ((num=n1; num<=n2; num++)); do
    echo -n "$num"

    if esPrimo $num; then
        echo " (Es Primo)"
    else
        echo ""
    fi
done

```

2.- (12 puntos) Escriba un *shell script* (**script2.sh**) que tome los datos contenidos en el archivo *Lista1.txt*, a continuación se muestra un fragmento de su contenido:

19990667	VIVANCO PRADA, MARIBEL
20020541	GASTELU ADVINCULA, JOSÉ MIGUEL
20030300	MARTÍNEZ LINARES, MARCO ANGELLO
20030555	CORDERO PRÍNCIPE, PIERRE ANDRÉS
20037170	BARDALEZ TORRES, IVAN
20040214	CÁCERES CHUMBITAZI, ANDREA SILVIA MARÍA
20040319	TASATO CÁNEPA, KENJY
20040428	HUALTA ESPINOZA, ELOISA CAROLINA
20042017	CHICANA VIVAR, WILDER JOSUE
20042023	CARRIÓN TELLO, JESÚS ANTONIO
20042057	GAMARRA STAGNARO, MIGUEL EDUARDO
20042129	MONGE VALDIVIESO, LUIS ALBERTO
20042291	SÁNCHEZ FIGUEROA, MARIELA ROSSANA
20044017	SALINAS MORALES, BERTHA KATHERINE
20044054	ONOFRE RIVERA, DAYSI PATRICIA
20047001	CARRANZA HERNANDEZ, SAMMY NAHIN
20047005	CANO CAUZO, JESUS HUMBERTO

El *script* debe obtener un nuevo archivo llamado *Lista2.txt* cuyo contenido, correspondiente al fragmento anterior, debe ser igual al que se presenta a continuación:

```
19990667:VIVANCO PRADA,Maribel:maribel.vivanco@pucc.edu.pe
```

```

##### Verifica existencia del archivo de entrada
# -f verifica que existe
if [[ ! -f Lista1.txt ]]; then
    echo "El archivo Lista1.txt no se encuentra en el directorio actual."
    exit 1
fi

##### Archivo de salida
if [[ -f Prueba.txt ]]; then
    #si existe, limpia
    > Prueba.txt
    echo "El archivo Prueba.txt ya existia, se limpio"
else
    #si no existe, lo crea
    touch Prueba.txt
    echo "El archivo Prueba.txt ha sido creado."
fi

##### Funcion para capitalizar
capitalizar(){

```

```

primeraVez=0
while IFS= read -r -n1 char #recorre todos los caracteres
do
    if [[ primeraVez -eq 0 ]]; then
        char=$(echo "$char" | tr '[:lower:]' '[:upper:]')
        primeraVez=1
    elif [[ $char == " " ]]; then
        primeraVez=0
    elif [[ primeraVez -eq 1 ]]; then
        char=$(echo "$char" | tr '[:upper:]' '[:lower:]')
    fi
    palabra+=$char
done <<< $1
echo $palabra
}

##### Procesar cada línea de Lista1.txt
# read -r line
while IFS= read -r line; do
    if [[ -z "$line" ]];then
        echo "Linea vacia, omitiendo"
    fi

    # Extraer código y nombres
    codigo="${line%%      *}" #del: tab
    cad="${line#*'\t'}" #del: tab
    apellidos="${cad%%,*}" #eliminina todo desp de ,
    nombres="${line##*,}"
    nombres="${nombres,,}"
    nombres=$(capitalizar "$nombres")
    usuario="${nombres%% *}}.${apellidos%% *}"

    # Formato para Lista2.txt
    echo "${codigo}:${apellidos},${nombres}:${usuario,,}@pucp.edu.pe" >> Prueba.txt
done < Lista1.txt

```

VERSION GH:

```

#!/bin/bash
# Hago el proceso linea por linea:
nombreGuardar="Lista2.txt"

while read LINE;
do

    cadena=$LINE
    num=${cadena:0:8}
    aux=${cadena:0:9}
    cadena=${cadena#$aux}

    IFS=' ' read -r -a array <<< "$cadena"

    variable=${array[0]}
    nuevoVariable="${variable// /}"

    #Ahora el formateo de los nombres:
    primerNombre=( "${array[2],,}" )
    primerNombre=( "${primerNombre^}" )

    segundoNombre=( "${array[3],,}" )
    segundoNombre=( "${segundoNombre^}" )
    # Para el correo:
    nombreCorreo=$primerNombre
    nombreCorreo=( "${nombreCorreo,,}" )

    apellidoCorreo=${array[0]}

```

```

apellidoCorreo=( "${apellidoCorreo,,}" )

echo "$num:${array[0]} ${array[1]} $primerNombre
$segundoNombre:$nombreCorreo.$apellidoCorreo@pucp.edu.pe" >> "$nombreGuardar"

done <"$1" #Ingreso el archivo como parámetro

```

3.- (4 puntos) Se tiene una lista de archivos que contienen un patrón en común. Se le solicita elaborar un *shell script* (**script3.sh**) para renombrar los archivos, eliminando el patrón en común. El patrón en común debe ser ingresado como argumento por la línea de comando.

Junto con el tema se le proporciona el archivo *archivos.zip* que contiene una lista de archivos para que pruebe su *script*.

```

alejandro@abdebian:~/Temas para el laboratorio 1/2022-2/Test$ ls
A10_202010_.txt.odt A12_202012_.txt.odt A4_202004_.txt.odt A7_202007_.txt.odt
A11_202011_.txt.odt A2_202002_.txt.odt A5_202005_.txt.odt A8_202008_.txt.odt
A1_202001_.txt.odt A3_202003_.txt.odt A6_202006_.txt.odt A9_202009_.txt.odt
alejandro@abdebian:~/Temas para el laboratorio 1/2022-2/Test$ ./script2.sh _
alejandro@abdebian:~/Temas para el laboratorio 1/2022-2/Test$ ls
A10_202010.odt A12_202012.odt A4_202004.odt A7_202007.odt
A11_202011.odt A2_202002.odt A5_202005.odt A8_202008.odt
A1_202001.odt A3_202003.odt A6_202006.odt A9_202009.odt
alejandro@abdebian:~/Temas para el laboratorio 1/2022-2/Test$ █

```

Los siguientes ejemplos le pueden ayudar:

```

stringZ=abcABC123ABcabc
echo ${stringZ/abc/xyz}      # xyzABC123ABcabc
                             # Replaces first match of 'abc' with 'xyz'.
echo ${stringZ//abc/xyz}     # xyzABC123ABcxyz
                             # Replaces all matches of 'abc' with 'xyz'.

echo .....
echo "$stringZ"              # abcABC123ABcabc
echo .....
                             # The string itself is not altered!

# Can the match and replacement strings be parameterized?
match=abc
repl=000
echo ${stringZ/$match/$repl} # 000ABC123ABcabc
echo ${stringZ//$match/$repl} # 000ABC123ABc000
# Yes!

echo

# What happens if no $replacement string is supplied?
echo ${stringZ/abc}          # ABC123ABcabc
echo ${stringZ//abc}         # ABC123ABC
# A simple deletion takes place.

```

```

#!/bin/bash
#Indico la carpeta donde se encuentran los archivos
carpeta="/home/osboxes/Desktop/LABORATORIO"

```

```

patron="$1"

```

```

for archivo in "$carpeta"/*; do
    if [ -f "$archivo" ]; then
        nombre=$(basename "$archivo")
        nuevo_nombre="${nombre//${patron}}" #Quito el patron del nombre del archivo
        mv "$archivo" "$carpeta/$nuevo_nombre" #Hago el reemplazo
    fi
done

```

NICK:

```

#!/bin/bash
# LA ANTIFIJA GENTE
malditosComunistas(){
    chain=$1 # LEE CADENA
    while read LINE #GUARDA LOS ARCHIVOS
    do # HACE UN DO
        nuevoNombre=${LINE//${chain}}
        #chainInicio="${LINE%%${chain}*" #ELIMINA SUFIJO MAYOR
        #chainFin="${LINE%*${chain}" #ELIMINA PREFIJO menor
        #nChain=${chainInicio}${chainFin} #CONCATENA LAS CADENAS
        nFile=$0 #GUARDA EL NOMBRE DEL ARCHIVO PERO CON ./
        nFile=${nFile#*/} #ELIMINA EL ./
        if [[ $LINE == $nFile ]]; then #COMPRUEBA QUE NO SEA EL SCRIPT
            continue #SI ES EL SCRIPT SALTA EL WHILE
        fi
        mv $LINE $nuevoNombre #CAMBIA NOMBRE
        #cp $LINE $nChain #HACE UNA COPIA DEL ARCHIVO
        #rm $LINE #ELIMINA EL ARCHIVO ANTERIOR
    done <<< $(ls) #LEE LOS DATOS DE CADA ARCHIVO
    # BUENAS NOCHES
}

```

```
}
malditosComunistas $1
```

LAB1 2024-1

1.- (5 puntos) Escriba un *shell script* (**script1.sh**) que reciba tres números. El primer número indica el inicio, el segundo número indica el final y el tercero es un dígito. El *script* debe imprimir todos los números entre inicio y final que contengan dicho dígito.

```
alejandro@abdebien:~/1/Temas para el laboratorio 1/2022-2$ ./script1.sh 100 130 2
102
112
120
121
122
123
124
125
126
127
128
129

alejandro@abdebien:~/1/Temas para el laboratorio 1/2022-2$ ./script1.sh 100 130 3
103
113
123
130
```

```
#!/bin/bash
x=$1
final=$2
patron=$3

while [ $x -le $final ]
do
    num=$x

    while [ $num -gt 0 ]
    do
        digito=$(( $num % 10 ))

        if [ $digito -eq $patron ]; then
            echo $x
            break
        fi

        num=`expr $num / 10`
    done

    x=`expr $x + 1`
done
```

3.- (8 puntos) Se tiene un archivo texto que tiene el siguiente contenido:

```
202001 NNNNNNNN 00000000, MMMMMMMM PPPPPPPP
202002 AAAAAAAAA BBBB BBBB EEEEEEE, CCCCCCCCC DDDDDDDD
202003 FFFFFFFF, GGGGGGGG HHHHHHHH
202004 IIIIIIII JJJJJJJ, KKKKKKK LLLLLLL
202005 MMMMMMMM NNNNNNNN, 00000000 PPPPPP
202006 QQQQQQQ, RRRRRRRR SSSSSSSS
202007 TTTTTTTT UUUUUU, VVVVVV WWWWWW
202008 XXXXXX YYYYYYYY ZZZZZZ, AAAAA BBBB
202009 NNNNNNNN 00000000, MMMMMMMM PPPPPPPP
202010 FFFFFFFF, GGGGGGGG HHHHHHHH
202011 IIIIIIII JJJJJJJ, KKKKKKK LLLLLLL
202012 TTTTTTTT UUUUUU, VVVVVV WWWWWW
202013 QQQQQQQ, RRRRRRRR SSSSSSSS
202014 AAAAAA BBBB BBBB EEEEEEE, CCCCCCCCC DDDDDDDD
202015 XXXXXX YYYYYYYY ZZZZZZ, AAAAA BBBB
```

Se le solicita escribir un *shell script* (**script3.sh**) para que el contenido se modifique de siguiente forma

```
Nnnnnnnn 00000000, Mnnnnnnnn Pppppppp 202001
Aaaaaaaaa Bbbbbbbb Eeeeeee, Cccccccc Dddddddd 202002
Ffffffff, Gggggggg Hhhhhhhh 202003
Iiiiiiii Jjjjjjj, Kkkkkkk Llllll 202004
```

```
$ cat intro
```

```
The UNIX operating system was pioneered by Ken
Thompson and Dennis Ritchie at Bell Laboratories
in the late 1960s. One of the primary goals in
the design of the UNIX system was to create an
environment that promoted efficient program
development.
```

```
$ tr '[a-z]' '[A-Z]' < intro
```

```
THE UNIX OPERATING SYSTEM WAS PIONEERED BY KEN
THOMPSON AND DENNIS RITCHIE AT BELL LABORATORIES
IN THE LATE 1960S. ONE OF THE PRIMARY GOALS IN
THE DESIGN OF THE UNIX SYSTEM WAS TO CREATE AN
ENVIRONMENT THAT PROMOTED EFFICIENT PROGRAM
DEVELOPMENT.
```

To capitalize first word only:

```
foo='one two three'
foo="${foo^}"
echo $foo
```

One two three

To capitalize **every word** in the variable:

```
foo="one two three"
foo-($foo) # without quotes
foo="${foo[@]^}"
echo $foo
```

One Two Three

```

while read LINE;
do
    cadena=$LINE

    cadena=( $cadena )
    cadena="${cadena[@]^}" #Le pongo mayúsculas a cada palabra

    num=${cadena:0:6} #extraigo el numero

    inicio="2020"?? #creo una cadena de la forma del numero

    cadena=${cadena#$inicio} #borro el numero del inicio
    echo "$cadena $num"

done <"$1" #Ingreso el archivo como parámetro

```

ALGUNAS FIJAS:

```

#!/bin/bash

fibonacci() {
    if [[ $1 -le 0 ]]; then
        echo 0
    elif [[ $1 -eq 1 ]]; then
        echo 1
    else
        echo $(( $(fibonacci $(( $1 - 1 ))) + $(fibonacci $(( $1 - 2 ))) ))
    fi
}

# Solicitar el número desde la consola
read -p "Ingrese un número para calcular Fibonacci: " num
echo "Fibonacci de $num: $(fibonacci $num)"
-

#!/bin/bash

fibonacci() {
    if [[ $1 -le 0 ]]; then
        echo 0
    elif [[ $1 -eq 1 ]]; then
        echo 1
    else
        echo $(( $(fibonacci $(( $1 - 1 ))) + $(fibonacci $(( $1 - 2 ))) ))
    fi
}

# Verifica que se haya pasado un argumento
if [[ -z $1 ]]; then
    echo "Uso: $0 <número>"
    exit 1
fi

echo "Fibonacci de $1: $(fibonacci $1)"

```

Euclides

```

#!/bin/bash

mcd() {
    if [[ $2 -eq 0 ]]; then
        echo $1
    else
        mcd $2 $(( $1 % $2 ))
    fi
}

```

```
# Verifica que se hayan pasado dos argumentos
if [[ -z $1 || -z $2 ]]; then
    echo "Uso: $0 <número1> <número2>"
    exit 1
fi

echo "MCD de $1 y $2: $(mcd $1 $2)"
se ejecuta: ./mcd.sh 56 98
```

Factorial, Invertir cadena, Sumatoria dígitos, Ordenar un array

```
factorial() {
    if [[ $1 -le 1 ]]; then
        echo 1
    else
        echo $(( $1 * $(factorial $(( $1 - 1 ))) ))
    fi
}
echo "Factorial de $1: $(factorial $1)"
-
factorial_iterativo() {
    resultado=1
    for (( i=1; i<=$1; i++ )); do
        resultado=$(( resultado * i ))
    done
    echo $resultado
}
echo "Factorial de $1: $(factorial_iterativo $1)"

####
invertir_cadena() {
    echo "$1" | rev
}

echo "Cadena invertida: $(invertir_cadena $1)"
#####
sumar_digitos() {
    num=$1
    suma=0
    while [[ $num -gt 0 ]]; do
        suma=$(( suma + num % 10 ))
        num=$(( num / 10 ))
    done
    echo $suma
}

echo "Suma de dígitos de $1: $(sumar_digitos $1)"
#####
ordenar_burbuja() {
    local arr=("$@")
    local n=${#arr[@]}

    for (( i=0; i<n-1; i++ )); do
        for (( j=0; j<n-i-1; j++ )); do
            if [[ ${arr[j]} -gt ${arr[j+1]} ]]; then
                temp=${arr[j]}
                arr[j]=${arr[j+1]}
                arr[j+1]=$temp
            fi
        done
    done

    echo "Array ordenado: ${arr[@]}"
}

ordenar_burbuja "$@"
```